

CARGOMATE

REAL-TIME INVENTORY TRACKING SYSTEMS WITH

BARCODE SCANNER

25J-26J-032

Project Proposal Report

S S Hewapathirana
IT22014122

BSc. (Hons) Degree in Information Technology, Specialized in
Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

August 2025

CARGOMATE

**REAL-TIME INVENTORY TRACKING SYSTEMS WITH
BARCODE SCANNER**

25J-26J-032

Project Proposal Report

BSc. (Hons) Degree in Information Technology, Specialized in
Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

August 2025

Declaration of the Candidate & Supervisor

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any previously published or written by another person material, except where the acknowledgement is made in the text.

Group Member Name	Student ID	Signature
Hewapathirana S S	IT22014122	

The above candidate are carrying out research for the undergraduate Dissertation under my supervision.


Signature of the Supervisor :

30/08/25
Date:

Abstract

Inventory management is a vital function for retail stores, warehouses, and supply chains, as it directly impacts cost efficiency, customer satisfaction, and overall business performance. Traditional inventory tracking methods often rely on manual processes or outdated systems, which are prone to errors, delays, and stock mismatches. These shortcomings can result in overstocking, understocking, and inaccurate reporting, ultimately leading to financial losses and inefficiencies. To address these challenges, this project focuses on the development of a real-time inventory tracking system using barcode technology.

In the proposed system, each product is labeled with a barcode that can be scanned using either a handheld device or a mobile application. Once scanned, the product details are instantly updated in a centralized database, enabling accurate and immediate reflection of stock-in and stock-out activities. This ensures that managers and staff always have access to the latest inventory levels without depending on manual data entry. The system is designed with a cloud-integrated database, allowing real-time access from multiple devices, as well as an offline mode that temporarily stores scanned data and synchronizes it once connectivity is restored.

Additional functionalities, such as low-stock alerts, error handling, and automated reporting, enhance the reliability and usability of the system. A user-friendly dashboard presents live inventory status, movement history, and generates analytical reports that help decision-makers identify fast-moving products, sales trends, and reorder needs. Compared to RFID systems, barcode-based tracking provides a cost-effective and practical solution, particularly for small and medium-sized businesses that require efficiency without heavy investment.

By reducing manual workload, minimizing errors, and providing accurate real-time visibility, the system improves inventory control and supports smarter business decisions. This solution aims to bridge the gap between traditional practices and modern digital inventory systems, contributing to higher accuracy, lower operational costs, and overall improvement in retail and warehouse efficiency.

Table of Contents

01. INTRODUCTION	1
1.1 Background Study	1
1.2 Literature Survey	1
1.3 Research Gap	3
1.4 Research Problem	5
02. OBJECTIVES	6
2.1 Main Objective	6
2.2 Specific Objectives	6
03. METHODOLOGY	7
3.1 System Overview Diagram	7
3.2 Components and Their Roles	9
3.4 Key Functionalities of Real-Time Inventory Tracking with Barcode	9
3.5 Implementation – Real-Time Inventory Tracking with Barcode	10
3.6 Feasibility Study	11
04. Project requirements	13
4.1 Functional Requirements	13
4.2 Non-Functional Requirements	14
4.3 System Requirements	15
4.4 Use Cases	16
4.5 Test Cases	17
05. PROJECT MILESTONES	18
06. WORK BREAKDOWN CHART	19
07. DESCRIPTION OF PERSONNEL AND FACILITIES	21
08. DESCRIPTION OF BUDGET	22
09. COMMERCIALIZATION	23
9.1 Target Market	23
10. REFERENCES	24

LIST OF FIGURES

Figure 1: Systems Diagram 7

Figure 2: Component Diagram 8

LIST OF TABLES

Table 1: Gap Chart 4

Table 2: Gantt Chart 18

Table 3: Budgett 22

01. INTRODUCTION

1.1 Background Study

In modern supply chain and warehouse management systems, real-time inventory tracking is an essential requirement to ensure operational efficiency and customer satisfaction. Traditional inventory management relies heavily on manual stock counting and paper-based or semi-digital systems, which are time-consuming, prone to human error, and often fail to provide accurate real-time visibility.

To overcome these challenges, industries have adopted technologies such as barcode-based tracking, RFID systems, and IoT-enabled cloud solutions. Barcodes remain one of the most cost-effective and widely adopted methods of item identification due to their simplicity, reliability, and low implementation cost. However, many barcode systems in current use are still manually operated and lack integration with automation platforms such as warehouse robots and cloud databases.

Thus, the need arises for a barcode-driven, real-time, automated inventory tracking system that can seamlessly integrate with robotic warehouse operations, provide instant cloud updates, and reduce dependence on manual processes

1.2 Literature Survey

1. Practical edge implementations

Edge capture is repeatedly shown to be both feasible and cost-effective with commodity hardware (phones, cameras, simple scanners) when paired with disciplined capture flows. Retail/warehouse prototypes report low latency and good accuracy as long as the scanning pipeline enforces stable framing, quick feedback, and transactional updates to the store database [1][6]. Real-time dashboards (even in non-web stacks like LabVIEW) demonstrate that near-instant visibility is achievable with deterministic loops and clear alerting thresholds (e.g., low-stock triggers) [5]. Smart shopping-cart systems extend the “edge” to moving platforms—scanning items in cart and immediately adjusting local counts—illustrating how continuous, on-device identification reduces shrink and stock-outs while keeping user interaction friction-low [8].

Implications: Design for first-pass reads (good lighting hints, angular tolerance) and immediate UI confirmation (beep/haptic) before committing inventory movements. Keep the edge logic simple and fast; defer heavy analysis to the back end.

2. Cloud-integrated barcode inventory systems

End-to-end IoT designs consistently publish barcode events (SKU, time, location/session, user/device) to a central service, where stock ledgers, alerting, and analytics live [1][6][7]. Cloud consolidation enables unified views (by aisle, shelf, or site) and supports downstream forecasting and planogram optimization—smart-cart architectures are a clear example linking scan streams to demand modeling [8]. Reviews emphasize modularity (device → gateway → message broker → services → dashboard) so components can evolve independently [7].

Implications: Adopt a clean event schema and idempotent “stock movement” API; maintain append-only logs for auditability and reconciliation.

3. Edge-to-cloud communication for real-time updates

Across the systems surveyed, low-overhead, near-real-time communication patterns (publish/subscribe messaging or lightweight REST) are favored to push scan events upstream with minimal delay [1][5][7]. Store/wearable devices can buffer briefly during connectivity blips, then flush in order, preserving event time stamps for correct ledger sequencing [7]. Where very tight freshness is needed (e.g., active picking), studies recommend throttling UI refresh intervals but not the event pipeline, so visibility remains “live” without overloading networks [5][7].

Implications: Use a message broker or event ingestion endpoint with backpressure handling; include device clock sync and sequence IDs to prevent double-counting.

4. On-device decoding and libraries

Algorithmic work shows robust, fast barcode pipelines on modest CPUs: pre-processing (denoise, adaptive threshold), region/edge detection, geometric correction for skew/perspective, and efficient decode heuristics—achieving real-time performance even under noise and partial occlusion [3]. For 2D codes captured at awkward angles (e.g., high shelves), perspective normalization and morphology-based proposals significantly raise detection rates before decoding [4]. These steps match well to mobile/embedded implementations where small compute budgets demand lean image pipelines.

Implications: Implement a two-stage approach: (1) fast region proposal + perspective fix; (2) decode with early-exit heuristics. Cache recent exposure/ROI settings to stabilize frame-to-frame reads.

5. Barcode vs. RFID for warehouses

Modeling and simulation comparing stand-alone barcode, stand-alone RFID, and hybrid pipelines highlight the trade-off space: barcodes win on hardware cost, ubiquity, and error transparency; RFID wins on unattended bulk reading and line-of-sight independence; hybrids can approach RFID-like throughput while retaining barcode cost advantages [2]. Survey work echoes this: RFID excels in bulk inventory sweeps and closed-loop tracking, while barcode

systems remain compelling for item-level transactions, audits, and low-capex deployments [7]. Smart-cart and IoT store systems often start barcode-first, then selectively add RFID to back-of-house or pallet flows where it dramatically reduces labor [8][2].

Implications: For your scope—shelf-level, real-time counts—barcode is a strong baseline. Consider targeted RFID for pallets/receiving to accelerate cycle counts without retooling front-of-house.

6. Barcode standards and symbologies

The papers distinguish between 1D codes (typical retail/warehouse: EAN/UPC, Code 128) and 2D codes (QR, Data Matrix) [3][4]. Algorithmic and drone/offset-capture studies focus on resilience to rotation, skew, and partial occlusion for 2D marks [4], while fast 1D pipelines emphasize line/edge localization and geometric correction before decoding [3]. In practice, warehouses frequently mix formats: 1D for fast, legacy scanner compatibility; 2D where denser data (batch/lot, expiration) is needed on small labels [7].

Implications: Support at least Code 128 + EAN/UPC (1D) and QR/Data Matrix (2D). Keep symbology auto-detect enabled, but bias to the site’s dominant formats to reduce decode time.

1.3 Research Gap

Current research on barcode-based inventory systems has shown progress in enabling real-time tracking of stock by automatically updating databases whenever items are scanned. While these approaches reduce manual errors and improve visibility, they still face several shortcomings. One major issue is that most systems rely on raw barcode inputs, meaning duplicate scans or invalid barcodes can still enter the system unchecked. This weakens inventory accuracy and requires human intervention to correct mistakes.

Another limitation lies in communication efficiency. Existing systems typically transmit every scanned record directly to the cloud, regardless of whether the information has changed. This approach leads to unnecessary bandwidth usage, increased latency, and higher operating costs, especially in large-scale retail or warehouse environments. A further gap is reliability under unstable network conditions. Since most IoT-integrated barcode systems depend on continuous internet connectivity, a loss of connection can halt updates and create discrepancies in stock records. This makes such systems vulnerable in areas with weak or inconsistent networks.

To address these limitations, this study introduces two key novelties. First, Adaptive Edge–Cloud Processing will allow the system to intelligently process data at the edge, using machine learning to filter out duplicate scans, detect anomalies, and send only validated updates to the cloud. Second, an Offline Cache with Auto-Sync will ensure that all scans are stored locally during disconnections and automatically synchronized once connectivity is restored. Together, these

innovations provide a barcode-based inventory system that is accurate, efficient, and resilient, overcoming the core challenges seen in existing research.

Table 1: Gap Chart

Features	Current Research	Identified Gap	Proposed Solution
Barcode-based Real-Time Tracking	Systems like LabVIEW-based tracking update stock instantly when products are scanned. [5]	Limited to raw scan input; cannot filter duplicate scans or detect invalid/fake barcodes.	Introduce Adaptive Edge-Cloud Processing with ML to detect anomalies and remove redundant scans.
IoT and Cloud Integration	IoT-enabled barcode systems use cloud platforms for inventory monitoring and decision-making	Require continuous internet connectivity; disconnections cause missing records	Add Offline Cache with Auto-Sync to store local data and synchronize automatically when the connection is restored.
Low-cost and widely adopted barcode systems	Widely used for product identification and checkout tracking in retail and warehouse operations [3], [5]	Dependent on manual scanning, leading to delays and human errors	Automate barcode capture using smart scanners

1.4 Research Problem

Managing inventory with accuracy and reliability remains a significant challenge in modern retail and warehouse environments. Although barcode-based systems are widely adopted for their simplicity and low cost, existing solutions still face critical issues that affect real-time performance. Current barcode tracking systems primarily depend on direct scanning and cloud synchronization, which makes them vulnerable to several problems. Duplicate or invalid scans often enter the system without detection, leading to inaccurate stock records. In addition, these systems typically transmit all scanned data to the cloud, even when no meaningful changes occur, resulting in unnecessary bandwidth usage and latency.

Another major limitation arises when network connectivity is unstable. Most barcode-based inventory solutions require a constant internet connection for real-time updates, meaning that any disruption causes gaps or delays in stock records. This lack of resilience can severely impact decision-making, particularly in fast-moving retail settings where accurate and timely data is essential.

Therefore, the core research problem is the lack of an intelligent and robust real-time barcode tracking system that can:

1. Detect and filter duplicate or invalid scans.
2. Optimize communication by transmitting only necessary data.
3. Ensure uninterrupted tracking even during internet outages.

Addressing this problem is crucial to improving the efficiency, accuracy, and reliability of barcode-based inventory management in real-world applications.

02. OBJECTIVES

2.1 Main Objective

To create a smarter warehousing automation solution that combines cloud-based inventory management, robotic put-away and picking, autonomous navigation, and QR-code-based product identification to augment operational efficiency, accuracy, and scalability of supply chain management.

2.2 Specific Objectives

1. Implement Barcode Scanner Integration - Develop a system that enables automatic item identification using 1D and 2D barcode scanners mounted on robotic arms or trolleys. Ensure compatibility with multiple barcode standards (e.g., UPC, EAN, QR, DataMatrix) for versatility in real-world warehouse environments. It provides reliable hardware-software interfacing between the scanner and an edge processing unit (Raspberry Pi/MCU).

2. Design Real-Time Data Processing and Synchronization - Process scanned data locally on the edge device to validate, filter, and detect duplicates before transmission. Implement a real-time communication pipeline using IoT protocols such as MQTT or HTTPS, ensuring that every scan is reflected instantly in the cloud database. Then can enable low-latency synchronization, with updates visible on the dashboard within seconds.

3. Develop Error Detection and Validation Mechanisms - Create algorithms to handle common issues such as duplicate scans, unreadable labels, and corrupted data inputs. Maintain a log of errors and retries for transparency and post-analysis. Improve the reliability of the system by ensuring that only valid and accurate inventory data is stored in the cloud database.

4. Integrate Cloud Database for Inventory Management - Design a centralized cloud database (Firestore/MySQL/AWS) to store real-time stock information. Define and implement data models for items, events (stock-in, stock-out, movement), and inventory levels. Ensure secure access control and authentication, preventing unauthorized manipulation of data.

5. Ensure the Scalability and Reliability of the System - Design the system architecture to support multiple robotic units and large-scale warehouse operations without performance degradation. Incorporate an offline-first approach where events are cached locally during network failures and automatically synced when connectivity is restored. Evaluate system performance in terms of accuracy, speed, and reliability, aiming for at least 99% data accuracy and real-time responsiveness.

6. Testing and Validation - Conduct unit testing for barcode reading, validation logic, and data transmission. Perform integration testing to ensure seamless communication between scanner, edge device, cloud, and dashboard. Carry out user acceptance testing (UAT) in a simulated warehouse environment to confirm system usability and practicality.

03. METHODOLOGY

3.1 System Overview Diagram

The system enables automated scanning of items using barcode readers mounted on robotic arms or mobile trolleys. Each scanned item is processed by a Raspberry Pi/MCU, which decodes and validates the data before transmitting it to the cloud database. The database stores stock-in, stock-out, and stock-move events, while a web-based dashboard provides real-time inventory visibility to warehouse operators.

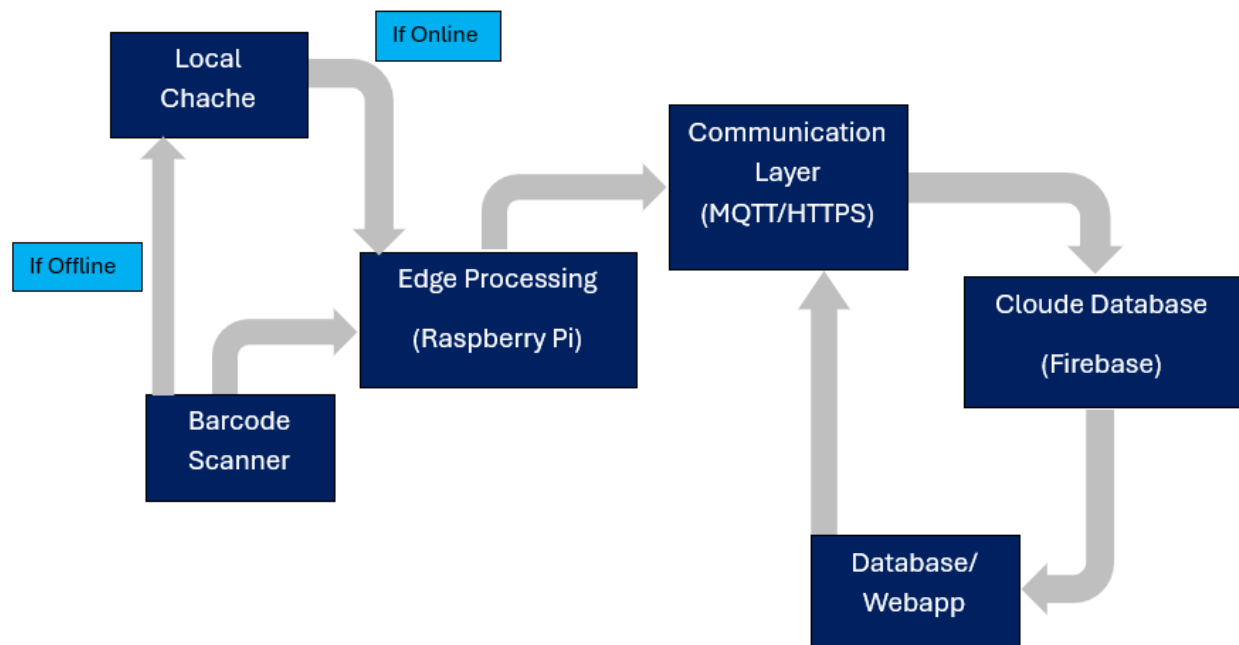


Figure 1: Systems Diagram

1. Barcode Scanner

This is the device used to scan barcodes from products. It can be a handheld scanner, a mobile device with a camera, or even an automated trolley/robotic arm in larger warehouses. Its job is to capture barcode data quickly and accurately.

2. Edge Processing (Raspberry Pi / MCU)

Once the barcode is scanned, the data first goes to an edge device such as a Raspberry Pi or microcontroller (MCU). This device checks and cleans the data before sending it further. It reduces errors and ensures that only valid data is processed, which also makes the system faster.

3. Communication Layer (MQTT / HTTPS)

This layer is responsible for sending the validated data to the cloud. Protocols like MQTT (lightweight IoT protocol) or HTTPS (secure web transfer) are used. It also handles two-way communication, meaning it not only sends inventory updates but can also receive alerts and user requests.

4. Cloud Database (Firebase / MySQL)

The cloud database stores all product details, stock levels, and history. Using a cloud-based system ensures that the data is safe, centralized, and accessible from anywhere. Options like Firebase (real-time cloud) or MySQL (relational database) can be used depending on system needs.

5. Dashboard / Web App (Live Stock, Alerts, Reports)

This is the user interface where managers and staff can see the inventory status in real time. It shows stock levels, generates alerts when stock is low, and produces reports for decision-making. This dashboard can be accessed on a computer or mobile device, making it convenient for users.

3.2 Component Diagram

The component diagram illustrates the major modules of the Real-Time Inventory Tracking System with Barcode Integration and how they interact to deliver smooth inventory management.

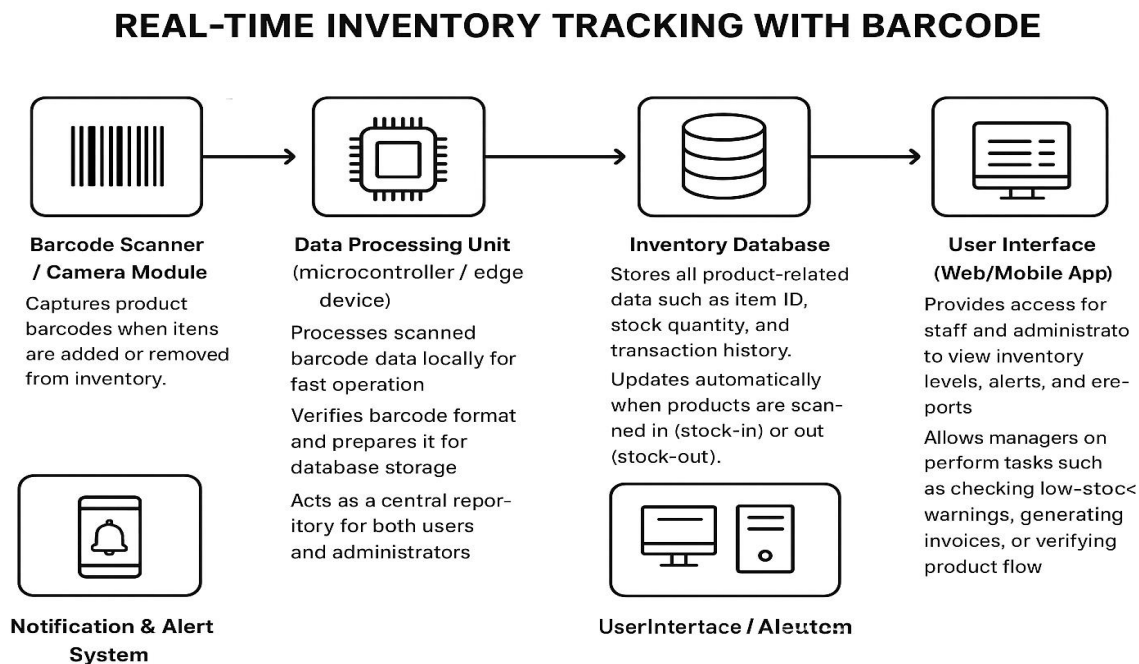


Figure 2: Component Diagram

3.2 Components and Their Roles

1. Barcode Scanner / Camera Module - Captures product barcodes when items are added or removed from inventory. This acts as the primary data input device, ensuring every transaction is recorded instantly.

2. Data Processing Unit (Microcontroller / Edge Device) - Processes the scanned barcode data locally for fast operation. Verifies the barcode format and prepares it for database storage. Ensures real-time performance by reducing delays before sending data to the cloud/server.

3. Inventory Database - Stores all product-related data such as item ID, stock quantity, and transaction history. Updates automatically when products are scanned in (stock-in) or out (stock-out). Acts as the central repository for both users and administrators.

4. Cloud / Server Backend - Manages heavy operations like report generation, demand forecasting, and secure data storage. Provides connectivity for remote monitoring and system scalability. Cloud ensures data synchronization across all connected devices in real-time.

5. User Interface (Web/Mobile App) - Provides access for staff and administrators to view inventory levels, alerts, and reports. Allows managers to perform tasks such as checking low-stock warnings, generating invoices, or verifying product flow. It ensures a user-friendly experience with real-time updates.

6. Notification & Alert System - Generates low-stock alerts and error notifications. Ensures timely action to avoid stockouts or overstock situations. Can be integrated with email, SMS, or in-app notifications.

3.4 Key Functionalities of Real-Time Inventory Tracking with Barcode

1. Barcode-Based Item Identification

Every product is labeled with a barcode. The system scans the barcode to uniquely identify the product and fetch its details such as name, category, price, and supplier.

2. Stock-In (Adding Items to Inventory)

When new stock arrives, products are scanned, and the system automatically increases the quantity in the database. For bulk items, one product can be scanned and total quantity entered, reducing time and effort.

3. Stock-Out (Removing Items from Inventory)

During sales or dispatch, scanning the product barcode automatically reduces the stock level in the database. This ensures that the inventory count always matches actual stock.

4. Real-Time Database Updates

Every scan instantly updates the database, ensuring managers always see the latest stock levels. This reduces the chances of overstocking or running out of items.

5. Low-Stock Alerts

The system monitors quantities and generates alerts when stock goes below a set threshold. This helps staff plan reordering on time and avoid shortages.

6. Error Handling & Manual Entry

If a barcode is damaged or unreadable, the system allows manual entry of the product ID to ensure the process doesn't stop.

7. Dashboard & Reports

A user-friendly dashboard displays current stock levels, items sold, and alerts. Managers can generate reports for sales trends, product movement, and inventory history.

8. Search & Tracking

Users can quickly search for products using barcode scans or item codes, making it easy to track product location and availability within the store or warehouse.

9. Security & Access Control

Staff logins ensure that only authorized users can add, remove, or edit stock details. This prevents misuse and keeps the inventory data secure.

3.5 Implementation – Real-Time Inventory Tracking with Barcode

1. Barcode Scanning and Data Capture

The implementation begins with barcode scanners, either handheld devices or mobile phones with scanning applications. Each product in the inventory is labeled with a barcode that uniquely identifies it. When scanned, the system decodes the barcode and retrieves the corresponding product details, such as name, ID, and category. This step ensures that every product movement, whether stock-in or stock-out, is recorded instantly without relying on manual entries.

2. Application Server and Business Logic

Once the barcode data is captured, it is processed by the application server, which acts as the brain of the system. The server checks the transaction type—whether items are being added to stock or removed during sales—and updates the records accordingly. It also validates the entries, preventing duplicate scans, incorrect product IDs, or negative stock levels. This logical layer ensures smooth operation and accuracy across the entire system.

3. Real-Time Database Management

The inventory database plays a central role in this implementation. It stores product details, stock quantities, and the complete history of transactions. Each scan updates the database in real time, allowing managers and staff to have an up-to-date view of inventory. In case of internet failures, the system is designed with an offline mode where data is temporarily stored on the device and synchronized with the cloud once connectivity is restored. This makes the solution both reliable and practical.

4. User Dashboard and Reporting

To make the system user-friendly, a dashboard interface has been designed. This interface allows staff and managers to monitor live stock levels, view sales history, and check movement logs. Low-stock alerts are automatically generated when quantities fall below a threshold, helping with timely reordering. In addition, the dashboard can generate analytical reports, giving insights into fast-moving items, stock trends, and overall efficiency of inventory operations.

5. Alerts and Notifications

A key feature of the system is its ability to generate alerts whenever critical conditions arise. For instance, if the stock of an essential product drops below the minimum required level, the system automatically notifies the user through the dashboard or via email/SMS. This ensures that stockouts are avoided and that replenishment can be planned before shortages occur.

6. Security and Access Control

To maintain data integrity, the system includes secure login and role-based access. Administrators have full control, including the ability to add or remove products and users, while staff can only perform scanning and basic stock operations. By restricting permissions, the system reduces the risk of errors or misuse and ensures that all transactions are recorded against verified user accounts.

3.6 Feasibility Study

1. Technical Feasibility

The proposed system is technically feasible as barcode technology is well-established, affordable, and easy to implement. Handheld barcode scanners or mobile applications with camera scanning can be used to capture product data. These devices are readily available and compatible with standard databases and applications. On the software side, the system will run on a simple client-server model, with a database to store product details and a web or mobile dashboard for users. Since all the required tools—such as barcode libraries, databases, and reporting frameworks—are accessible and well-documented, there are no major technical barriers to implementation.

2. Economic Feasibility

From a financial perspective, the system is cost-effective because barcode scanners are cheaper than alternatives like RFID. The only costs involved are hardware (scanners or smartphones), software development, and hosting a cloud database. These are one-time or low-maintenance expenses. The benefits, however, are significant. The system reduces manual work, prevents stock-out losses, minimizes errors, and ensures real-time visibility of stock. Over time, the cost savings from reduced waste, improved accuracy, and better decision-making will outweigh the initial investment, making the system economically viable.

3. Operational Feasibility

The system is highly practical for daily use in retail or warehouse environments. Staff members can easily adapt to scanning products instead of manually entering stock numbers. The dashboard provides a simple interface to view live stock, generate reports, and receive alerts when items run low. Because the system is designed to be user-friendly, training requirements are minimal, and existing staff can operate it without specialized skills. By automating tedious tasks and providing real-time data, the system directly supports smoother operations and higher efficiency.

4. Schedule Feasibility

The project is also feasible within the proposed timeframe. The system is divided into manageable phases such as literature review, design, prototype development, integration, testing, and final deployment. Each phase can be realistically completed within the one-year project timeline. The use of existing barcode libraries and open-source tools will further shorten development time, ensuring that the system can be implemented without delays.

04. Project requirements

4.1 Functional Requirements

1. Barcode Scanning and Identification

The system should be able to scan product barcodes using a handheld device or a mobile application. Each scanned code must be correctly identified and matched with its product details, such as name, category, and supplier. This ensures that every item entering or leaving the inventory is tracked automatically, avoiding mistakes that usually occur in manual entry.

2. Stock-In Management

When new stock arrives, the system should allow the user to scan the products and record them as part of the inventory. The product quantity must automatically increase in the database, reflecting the added stock. For bulk products, the system should also allow scanning one item and entering the total quantity to save time and reduce effort.

3. Stock-Out Management

During sales or dispatch, the system should record the removal of products by scanning their barcodes. Each scan should reduce the quantity of that item in the database instantly. The system must also prevent negative stock levels, ensuring that products cannot be sold or dispatched beyond the available quantity.

4. Real-Time Database Updates

The system should update the central database immediately after each scan. This allows managers and staff to always see the correct inventory levels without delay. Real-time updates reduce errors and ensure that stock records reflect the actual quantities available in the warehouse or store.

5. Dashboard and Reporting

The system should provide a user-friendly dashboard that displays current stock levels, recent transactions, and alerts. It should also generate reports on stock movement, sales trends, and product history. These reports will help managers make better decisions regarding purchasing and stock planning.

6. User Management and Security

The system should allow different user roles, such as administrators, staff, and viewers, each with specific permissions. Administrators can manage products and users, staff can update stock using barcode scanning, and viewers can only check reports. This ensures secure use of the system and prevents unauthorized changes to critical data.

4.2 Non-Functional Requirements

1. Performance

The system should perform all inventory updates in real-time. Each barcode scan must be processed and reflected in the database within a few seconds, ensuring that users always have access to accurate and up-to-date information. The system should also be able to handle multiple users scanning items at the same time without slowing down.

2. Reliability

The system must be dependable and operate without frequent crashes or errors. It should continue to function even if internet connectivity is temporarily lost by storing scanned data locally and syncing it once the connection is restored. This ensures that inventory tracking is never interrupted.

3. Scalability

The system should be flexible enough to grow with the needs of the business. As more products, users, or warehouses are added, the system should handle the increased load without performance issues. This makes the solution suitable not only for small stores but also for larger retail chains.

4. Usability

The interface should be simple and easy for staff to use with minimal training. Buttons, menus, and dashboards must be designed to be clear and intuitive so that users can quickly perform stock-in and stock-out operations without confusion.

5. Security

The system must ensure that all inventory data is stored securely. User authentication and role-based access should be enforced so that only authorized staff can make changes. Sensitive information, such as login credentials, should be encrypted to prevent unauthorized access.

6. Maintainability

The system should be easy to maintain and update. Regular software updates, bug fixes, and improvements should be possible without disrupting ongoing operations. This ensures that the system can adapt to future needs and remain efficient over time.

7. Compatibility

The system should be compatible with standard barcode scanners, mobile devices, and cloud databases. It should also integrate smoothly with optional systems like Point-of-Sale (POS) or Enterprise Resource Planning (ERP) software if needed in the future.

4.3 System Requirements

1. Hardware Requirements

To run the real-time inventory tracking system smoothly, basic hardware support is required. A barcode scanner or mobile device with a camera will serve as the main input device. The system also requires a server or computer to host the database and application. For smaller setups, a standard desktop computer with at least 8GB RAM, a multi-core processor, and 500GB storage is sufficient. In larger environments, cloud servers can be used for better performance and scalability. Reliable internet connectivity is also important to keep the system updated in real-time.

2. Software Requirements

The software environment should include an operating system such as Windows, Linux, or Android (for mobile scanning). The backend can be built using technologies like Python, Java, or Node.js, while the database can be MySQL, Firebase, or any other SQL/NoSQL option. The frontend will be a web dashboard or mobile app designed with a user-friendly interface. Additionally, barcode decoding libraries (e.g., ZXing, ZBar) will be required for scanning and identification.

3. Network Requirements

Since the system is intended to provide real-time updates, a stable network connection is crucial. Wi-Fi or 4G/5G connectivity should be available for mobile devices and scanners. The network must be secure, using HTTPS or MQTT protocols, to ensure safe communication between scanners, the database, and the dashboard. Offline syncing should also be supported so that the system can continue working even during temporary network failures.

4. Security Requirements

To protect sensitive inventory and sales data, the system must have secure login and user authentication. Role-based access control should ensure that only authorized personnel can update or delete records. Data encryption should be applied to communication between devices and the server, protecting the system from unauthorized access or cyber threats.

5. User Requirements

The system should be easy to learn and use for staff and managers. A simple dashboard with live updates, clear stock alerts, and straightforward scanning options is essential. The interface should require minimal training so that even non-technical staff can handle daily operations without difficulty. Users should also be able to generate reports, search for products, and check stock history conveniently.

4.4 Use Cases

UC-01: User Login & Role Access

Actor logs in with username and password to access the system; the system verifies credentials and loads permissions based on role (Admin, Staff, Viewer). Preconditions: user is registered.

Main flow: user enters credentials → system authenticates → role-based menus appear.

Postconditions: user session starts with allowed actions only; invalid attempts are rejected with an error message and no access.

UC-02: Stock-In (Receiving Items)

Staff scans a product's barcode when new stock arrives, enters quantity (or scans repeatedly), and confirms the receipt; the system validates the SKU, increments on-hand quantity, and records a movement log with time, user, device, and notes. Preconditions: product exists in the catalog. Postconditions: inventory increases and the transaction is visible on the dashboard and reports.

UC-03: Stock-Out (Sales/Dispatch)

Staff scans a product at dispatch or sale to reduce inventory; the system checks available quantity, prevents negative stock, decrements on-hand, and writes a movement record.

Preconditions: item exists and has sufficient stock. Postconditions: updated quantity is reflected instantly; if the action would go below zero, the system blocks it and shows an "insufficient stock" alert.

UC-04: Bulk Receiving (Fast Mode)

When pallets or cartons arrive, staff scans one representative barcode, enters the number of cartons and units per carton, and confirms; the system calculates the total units, updates stock in a single atomic operation, and logs the bulk receipt with references (GRN or supplier).

Preconditions: pack factor known or entered. Postconditions: large quantities are added quickly with a single auditable entry.

UC-05: Product Lookup & Details

Any user searches by barcode scan or text (SKU, name) to view a product card with current quantity, location, reorder level, and movement history. Preconditions: product exists.

Postconditions: user sees up-to-date data, can decide on actions (reorder, transfer, adjust) according to their role.

UC-06: Cycle Count / Inventory Audit

Auditor selects a counting list (by aisle/category), scans items on the shelf, and enters the physical count; the system compares physical vs. system quantity, flags variances, and lets an Admin approve an adjustment with a reason code. Preconditions: counting list generated.

Postconditions: discrepancies are resolved into adjustment movements with full audit trail.

UC-07: Reporting & Exports

Manager opens Reports, sets a date range and filters, and generates outputs like stock valuation, fast/slow movers, and movement logs; reports can be exported to CSV/PDF. Preconditions: data exists for the chosen period. Postconditions: report is generated, saved, and optionally shared for management decisions.

UC-08: Offline Scanning & Sync

When network is down, the mobile app queues scans locally with timestamps and a unique temp ID; once online, it syncs in order, the server validates each entry, de-duplicates using IDs, applies movements, and returns a sync summary. Preconditions: device storage available. Postconditions: no scans are lost; the dashboard reflects all synced changes

4.5 Test Cases

Test Case 1: User Login

This test ensures that only authorized users can access the system. The tester enters a valid username and password, and the system should allow access to the dashboard. If invalid credentials are provided, the system must display an error message and deny access. This case verifies the correctness of authentication and role-based access.

Test Case 2: Product Registration

In this test, a new product is added by scanning its barcode and entering details such as name, category, and quantity. The system should save the product successfully and display it in the inventory list. If the barcode already exists, the system must update the quantity instead of creating a duplicate record.

Test Case 3: Stock-In Operation

When new stock arrives, the staff scans the barcode of the product and enters the received quantity. The system should increase the stock count in real-time. This test confirms that database updates happen correctly and no errors occur during multiple scans of the same item.

Test Case 4: Stock-Out Operation

During sales or dispatch, the product is scanned, and its quantity is reduced automatically. The system must check that the stock level never goes negative. If a staff member tries to remove more items than available, the system should issue an “insufficient stock” alert.

Test Case 5: Product Search and Lookup

A product is searched by entering its barcode or name. The system should display the item’s details, including available stock, location, and movement history. This test validates the search accuracy and quick retrieval of records from the database.

05. PROJECT MILESTONES

Table 2: Gantt Chart

Barcode System		2025					2026						
Task	Month	8	9	10	11	12	1	2	3	4	5	6	7
✓ Literature Review & Background Study													
Requirement Analysis & System Design													
Barcode Research & Technology Selection													
Prototype Development													
System Integration & Database Setup													
Testing													
Documentation & Report writing													
Final Presentation and Submission													

06. WORK BREAKDOWN CHART

Phase 1 – Literature Review & Background Study

This phase focuses on understanding existing research, technologies, and methodologies related to inventory tracking with barcodes. It helps identify gaps, establish the research foundation, and justify the proposed system.

Phase 2 – System Architecture & Design

Here, the system's blueprint is created, including database schema, architecture diagrams, and workflow models. This phase ensures clarity on how different components (hardware, software, and databases) will interact.

Phase 3 – Barcode Scanner Hardware Selection & Integration

In this stage, the most suitable barcode scanner hardware is identified and procured. The scanner is integrated with the software system, ensuring smooth data capture and addressing error handling mechanisms.

Phase 4 – Real-Time Data Processing

This phase implements the core logic for real-time inventory updates. Data filtering, validation, and offline-first synchronization are added to ensure that the system remains accurate and reliable under different conditions.

Phase 5 – Cloud Database Integration & Security

Focuses on connecting the system with cloud databases for scalability and reliability. Security measures such as authentication, encryption, and controlled access are implemented alongside stock update automation.

Phase 6 – Dashboard Development & User Interface

A user-friendly dashboard is built to visualize stock levels, generate alerts, and provide analytics. This phase emphasizes UI/UX design so that users can easily monitor and interact with the system.

Phase 7 – Testing & Quality Assurance

Unit testing, integration testing, and error handling tests are performed to verify system reliability. User Acceptance Testing (UAT) ensures that the system meets real-world operational needs.

Phase 8 – Progress Presentations (I & II)

During this stage, interim progress presentations are delivered. These sessions ensure continuous feedback from supervisors and stakeholders, helping align the project with expectations.

Phase 9 – Final Report Writing & Documentation

The project outcomes, results, and evaluations are compiled into a structured final report. This phase also includes formatting and preparation of supporting materials for assessment.

Phase 10 – Final Presentation & Viva

The last stage focuses on preparing the final presentation, viva defense, and Q&A practice. It ensures that the research and implementation are communicated effectively to evaluators

07. DESCRIPTION OF PERSONNEL AND FACILITIES

Facilitators

- **Prof. Anuradha Jayakody** – Sri Lanka Institute of Information Technology (SLIIT)
- **Mr. Uditha Dharmakeerthi** – Sri Lanka Institute of Information Technology (SLIIT)

08. DESCRIPTION OF BUDGET

Table 3: Budgett

Component	Estimated Cost (LKR)
Barcode Scanner (USB / Basic Model)	6,500.00
Barcode Scanner (USB / Basic Model)	4,500.00
Cloud Database (Free Tier)	2,000.00
Testing Materials	1,500.00
Internet & Hosting	3,000.00
Report & Documentation	2,500.00
Software Tools	0.00

09. COMMERCIALIZATION

The target market for the system primarily includes retail stores, supermarkets, and warehouses where real-time stock visibility is essential. Small and medium-sized enterprises (SMEs) are particularly attractive customers because they often rely on manual inventory tracking, which is time-consuming and error-prone. By adopting this system, SMEs can achieve automation without the high investment costs of RFID. Large supermarket chains, distribution centers, and e-commerce fulfillment hubs also represent significant opportunities. These businesses handle thousands of products daily and require fast, accurate tracking systems to reduce stockouts and overstock situations. Beyond retail, the logistics industry—including shipping, wholesale distribution, and third-party warehousing—can also benefit from the system’s ability to provide instant updates and better stock control.

Overall, the system is designed to serve a wide market: from small retailers looking for affordable automation to large-scale enterprises seeking scalable, real-time tracking across multiple locations.

9.1 Target Market

Target Audience

- Small and Medium-Sized Retailers
- Supermarkets and Large Retail Chains
- Warehouses and Distribution Centers
- E-commerce Fulfillment Hubs
- Logistics and Supply Chain Companies

10. REFERENCES

- [1] V. Saillaja, K. Machap, M. Menaka, and V. Kumaravel, “Development of an IoT-based Inventory Management System for Retail Stores,” *Proc. Int. Conf. on Sustainable Computing and Smart Systems (ICSCSS)*, Coimbatore, India, 2023, pp. 954–958, doi: 10.1109/ICSCSS57650.2023.10169810.
- [2] N. T. N. Ton, T. T. Lam, M. T. Le, and T. D. Do, “Inventory Management System Using RFID and Barcode — A Modeling and Simulation Approach,” *Proc. 7th Int. Conf. on Green Technology and Sustainable Development (GTSD)*, 2024, pp. 362–368, doi: 10.1109/GTSD62346.2024.10675053.
- [3] O. Yang, “Algorithm Research and Implementation of Fast Identification of Barcode,” *Proc. 7th Int. Conf. on Computer Science & Education (ICCSE)*, Melbourne, Australia, Jul. 2012, pp. 746–748, doi: 10.1109/ICCSE.2012.6295120.
- [4] H. Cho, D. Kim, J. Park, K. Roh, and W. Hwang, “2D Barcode Detection using Images for Drone-assisted Inventory Management,” *Proc. 15th Int. Conf. on Ubiquitous Robots (UR)*, Honolulu, HI, USA, Jun. 2018, pp. 461–466, doi: 10.1109/URAI.2018.8441805.
- [5] S. N. Kumar and K. B. Ramesh, “Developing Real-Time Inventory Status System Using LabVIEW,” *Proc. 8th Int. Conf. on Computational Systems and Information Technology for Sustainable Solutions (CSITSS)*, 2024, pp. 1–6, doi: 10.1109/CSITSS64042.2024.10816824.
- [6] S. C. Roosevelt, E. Veemaraj, and S. Kirubakaran, “Real Time Stock Inventory Management System,” *Proc. 8th Int. Conf. on Inventive Systems and Control (ICISC)*, Coimbatore, India, 2024, pp. 156–163, doi: 10.1109/ICISC62624.2024.00034.
- [7] Smitha and A. Aslekar, “IoT in Inventory Management,” *Proc. Int. Conf. on Decision Aid Sciences and Applications (DASA)*, 2022, pp. 1045–1049, doi: 10.1109/DASA54658.2022.9765137.
- [8] M. I. Zulfiqar, A. Khalid, A. Siddig, M. J. Nawaz, and S. Saay, “AI-Driven Smart Shopping Carts with Real-Time Tracking and Inventory Forecasting for Enhanced Retail Efficiency,” *IEEE Access*, vol. 13, pp. 55576–55587, Mar. 2025, doi: 10.1109/ACCESS.2025.3553854.